

Informe — FNBC y desnormalización controlada

TP6 · Unidad 4 · Food Store · PostgreSQL 18

Entregable 1 01_tp_fnbc_control_lote.sql

Entregable 2 02_tp_desnormalizacion_top_categorias.sql

Ambiente

00_setup_ambiente.sql

Parte 1 — FNBC sobre control_lote_almacen

a) Dependencias funcionales

```
LoteID, DepositoID -> ResponsableControlID (trivial de la clave primaria)
ResponsableControlID -> DepositoID (dato maestro: cada responsable pertenece a un único depósito)
```

b) Clausuras y claves candidatas

Subconjunto	Clausura	¿Superclave?
{LoteID}	{LoteID}	No
{DepositoID}	{DepositoID}	No
{ResponsableControlID}	{ResponsableControlID, DepositoID}	No
{LoteID, DepositoID}	{LoteID, DepositoID, ResponsableControlID}	Sí (clave)
{LoteID, ResponsableControlID}	{LoteID, ResponsableControlID, DepositoID}	Sí (clave)
{DepositoID, ResponsableControlID}	{ResponsableControlID, DepositoID}	No

Claves candidatas: (LoteID, DepositoID) y (LoteID, ResponsableControlID). **Primos:** los tres atributos (cada uno pertenece a una clave candidata). **No primos:** ninguno.

c) ¿Cumple FNBC?

FNBC exige que toda DF no trivial tenga su determinante como superclave. La violatoria es `ResponsableControlID → DepositoID`: su determinante no es superclave (su clausura no contiene LoteID). Por lo tanto **no cumple FNBC** — aunque cumple 3FN porque DepositoID es atributo primo.

d) Anomalías de la instancia (501,30,801)·(502,30,801)·(503,31,802)

- **Inserción:** no se puede cargar un responsable nuevo (ej. 803) con su depósito si aún no controla un lote: LoteID (parte de la PK) no puede ser nulo.
- **Borrado:** anular el lote 503 pierde la única ocurrencia que registra que 802 pertenece al depósito 31 (borrar los 2 lotes de 801 perdería su depósito 30).
- **Actualización:** si 801 cambia de depósito hay que tocar las 2 filas (501,30) y (502,30); actualizar una sola deja al mismo responsable en dos depósitos.

e) Descomposición sin pérdida

- `lote_responsable(LoteID, ResponsableControlID)` — PK (LoteID, ResponsableControlID)
- `responsable_deposito(ResponsableControlID, DepositoID)` — PK ResponsableControlID

- Ambas con FK a lote/usuario/deposito y vista `v_control_lote_almacen` (reunión natural).

f) Unión sin pérdida y migración verificada

El atributo común de la descomposición, **ResponsableControlID**, es **superclave de R2** (su PK). Por el criterio de superclave, $R1 \bowtie R2$ reconstruye la relación original sin tuplas espurias. Migración ejecutada y verificada en el motor:

psql · verificación de equivalencia (EXCEPT)

```
1) en vista y NO en original    --- (0 filas)
2) en original y NO en vista    --- (0 filas)
```

Parte 2 — Desnormalización controlada del panel (top 5 del día)

Corrección del filtro de fecha. La consulta del enunciado filtra `ped.fecha = CURRENT_DATE`: fecha es `TIMESTAMP TZ` y `CURRENT_DATE` es `DATE`, la igualdad solo matchea la medianoche exacta (ejecución real con `actual rows=0` pese a tener 400 pedidos del día). Se corrigió a `fecha >= CURRENT_DATE AND fecha < CURRENT_DATE + 1` (mismo plan, mismo problema de reunión, reporte real devuelto).

a) PLAN ANTES (4 tablas) — evidencia/explain_antes.txt

food_store_tp3=> EXPLAIN (ANALYZE, BUFFERS) · consulta original

```
Limit  (cost=696.27..696.28 rows=4 width=40) (actual rows=4.00 loops=1)
 Buffers: shared hit=5241
-> Sort  (cost=695.86..696.23 rows=4 width=40) (actual rows=4.00)
-> GroupAggregate  (cost=695.86..696.23) Group Key: c.nombre
-> Nested Loop  (categoria) ... Rows Removed by Join Filter: 1768
-> Nested Loop  (detalle_pedido -> producto)
    -> Bitmap Heap Scan on pedido  (400 filas del día)
    -> Index Scan using idx_detalle_pedido  (400 loops, 1.608 buffers)
    -> Index Scan using producto_pkey  (1.205 loops, 3.615 buffers)
Planning Time: 56.121 ms
Execution Time: 23.586 ms
```

Nodo dominante: la cadena **Nested Loop** con los **Index Scan por fila** sobre `detalle_pedido` (400 loops) y `producto_pkey` (1.205 loops, 3.615 buffers). Total: 5.241 buffers tocados en **cada** ejecución del panel para resumir 1.205 líneas del día.

b) Patrón elegido: vista materializada (+ REFRESH CONCURRENTLY)

- **Evidencia que motiva la decisión:** el 65% de los buffers del plan (a) está en las uniones detalle→producto por fila; la MV elimina ese trabajo de la lectura y lo paga una vez en el refresco.
- **Mecanismo anti-desincronización:** la MV es proyección 100% derivada de las 4 tablas (filtro de baja lógica embebido) y se refresca con `REFRESH MATERIALIZED VIEW CONCURRENTLY` (índice único), programado, sin bloquear lecturas.
- **Reversibilidad sin pérdida:** `DROP MATERIALIZED VIEW` elimina el dato redundante sin tocar el origen. Frente al trigger (sumas en cada INSERT/UPDATE/DELETE de detalle, contención de escritura), la MV desacopla lectura y escritura.

d) PLAN DESPUÉS (MV) — evidencia/explain_despues.txt

food_store_tp3=> EXPLAIN (ANALYZE, BUFFERS) · consulta sobre la MV

```
Limit  (cost=14.00..14.01 rows=4 width=16) (actual rows=4.00 loops=1)
 Buffers: shared hit=9
```

```
-> Sort (cost=14.00..14.01 rows=4 width=16) (actual rows=4.00)
-> Bitmap Heap Scan on mv_top_categorias_dia (4 filas)
    -> Bitmap Index Scan on uq_mv_top_categorias_dia
        Index Cond: (dia = CURRENT_DATE)
Planning Time: 2.236 ms
Execution Time: 0.134 ms
```

Métrica	Antes (4 tablas)	Después (MV)
Tiempo de ejecución	23.586 ms	0.134 ms (~176× más rápido)
Buffers tocados	5.241	9
Nodo dominante	Nested Loop + Index Scan por fila (detalle 400×, producto 1.205×)	Bitmap Index Scan uq_mv_top_categorias_dia (4 filas)

e) Auditoría de desincronización (resultado esperado: vacío)

```
psql - script de auditoría (FULL JOIN + IS DISTINCT FROM contra la fuente)

dia | categoria_id | mv_total | fuente_total | tipo_desync
-----+-----+-----+-----+-----
(0 filas)
```

Capturas completas en TP6/evidencia/. Scripts: 01_tp_fnbc_control_lote.sql, 02_tp_desnormalizacion_top_categorias.sql, 00_setup_ambiente.sql.